Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 22 - PAO 2024 – II

Agenda

- Lección # 3
- Búsqueda heurística (o búsqueda informada)
  - Hill Climbing
  - Simulated Annealing
  - Best – First Search
    - Fronteras
    - Ejemplos
    - Teorema de Admisibilidad
    - Algoritmo de Backtracking

Hill Climbing

- ***Hill Climbing***
  - Es una versión mejorada del algoritmo de generación y prueba, pero con heurística
  - Desde el procedimiento de prueba, hay una retroalimentación para ayudar al generador de soluciones decidir, en qué dirección ir a continuación o si es necesario cambiar de dirección de búsqueda en el espacio de soluciones.
  - En este proceso, la búsqueda finaliza si no hay un próximo estado o un nodo razonable para moverse.

Simple Hill Climbing

1. Evaluar el estado inicial.
2. Repetir, hasta que se encuentre una solución, o ya no haya nuevos operadores que aplicar:

   - Seleccionar un nodo candidato y aplicar la función heurística

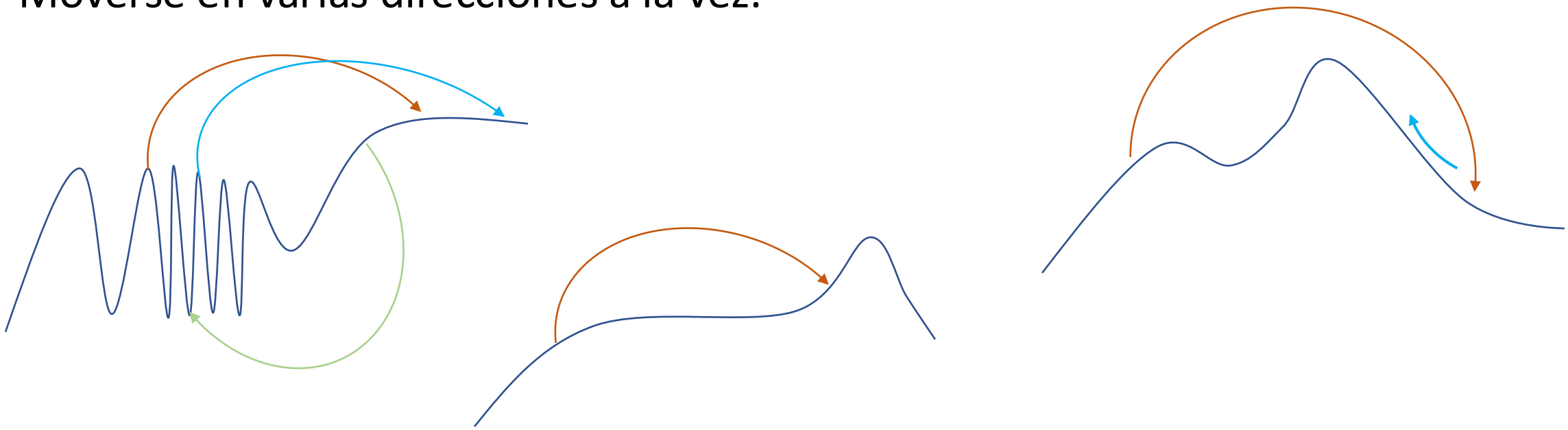
   - Evaluar el nuevo estado:

     - nuevo estado = objetivo ? $\rightarrow$ terminar
     - nuevo estado es mejor que el estado actual ? $\rightarrow$ avanzar con el nuevo estado actual

Hill Climbing: Desventajas

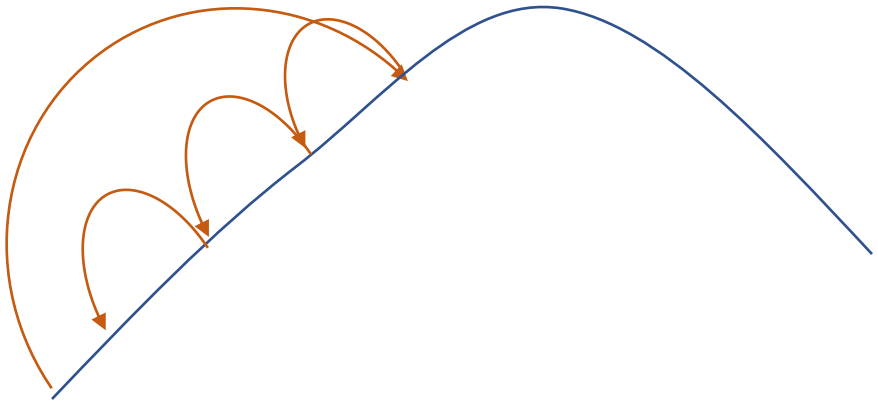
Salidas a estos problemas:

- Retroceder a un nodo anterior e intentar ir en una dirección diferente.
- Dar un gran salto para intentar ingresar a una nueva sección del espacio de soluciones.
- Moverse en varias direcciones a la vez.



Simulated Annealing

- El recocido simulado es una versión mejorada del algoritmo Hill Climbing.
- Al principio, este algoritmo explora una buena parte del espacio de la solución, de tal manera que la solución final podría no verse afectada por el estado inicial.
- Como consecuencia, la posibilidad de quedar atrapado en un máximo local, meseta o acantilado, es menor.



Simulated Annealing

1. Evaluar el estado inicial.
2. Repetir hasta que se encuentre una solución o no haya nuevos operadores para aplicar:

   - Establecer T como un estado inicial

   - Selecciona y aplica un nuevo operador

   - Evaluar el nuevo estado:

     goal $\rightarrow$ terminar

     ΔE = Valor (estado actual) - Valor (nuevo estado)

     $\Delta E < 0 \rightarrow$ nuevo estado actual

     else $\rightarrow$ nuevo estado actual con probabilidad $e^{-\Delta E/kT}$

Best-First Search

- **Best - First**
  1. **Lista open**: contiene los nodos generados.

     A los nodos se les ha aplicado la función heurística; pero aún no han sido probados; es decir, sus sucesores aún no se han generado. Esta cola puede considerarse como una cola de prioridad, en la que los elementos con mayor prioridad son los que tienen valores más prometedores, asignados por la función heurística.
  2. **Lista close**: contiene los nodos que se han examinado y no son el objetivo esperado.
  3. **Función heurística**: permite que el algoritmo busque primero las rutas más prometedoras.

Best-First Search

Función heurística:

$$f(n) = g(n) + h(n)$$

- g(n) = costo de la mejor ruta desde el estado inicial al nodo n.
- h(n) = costo de la mejor ruta desde el nodo n al estado objetivo.

Best-First Search

- **Best - First**
  - La función heurística ***f*** se puede definir como la suma de otras dos funciones, llamadas ***g*** y ***h***.
  - La función ***g*** es el costo para alcanzar el estado actual desde el estado inicial.
  - La función ***h*** es el costo estimado para alcanzar el estado objetivo desde el estado actual.

$$f(n) = g(n) + h(n)$$

Best-First Search

```
function best_first_search;

begin
   open := [Start];                                              % initialize
   closed := [ ];
   while open ≠ [ ] do                                           % states remain
      begin
         remove the leftmost state from open, call it X;
         if X = goal then return the path from Start to X
         else begin
                  generate children of X;
                  for each child of X do
                  case
                        the child is not on open or closed:
                           begin
                                 assign the child a heuristic value;
                                 add the child to open
                           end;
                        the child is already on open:
                           if the child was reached by a shorter path
                           then give the state on open the shorter path
                        the child is already on closed:
                           if the child was reached by a shorter path then
                              begin
                                 remove the state from closed;
                                 add the child to open
                              end;
                  end;                                           % case
                  put X on closed;
                  re-order states on open by heuristic merit (best leftmost)
               end;
return FAIL                                                      % open is empty
end.
```

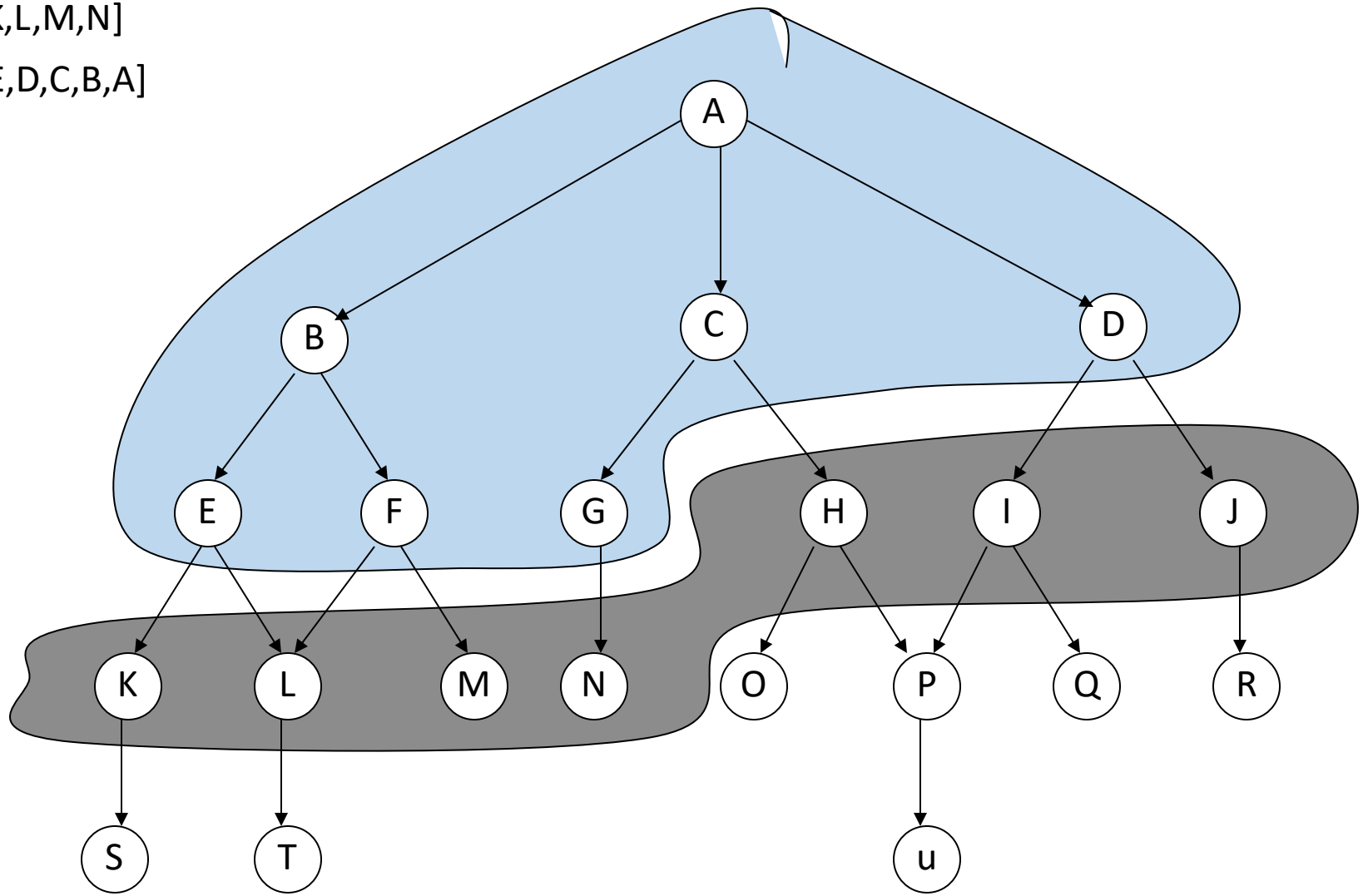
Fronteras

- Se refiere a la determinación de los límites o bordes de la búsqueda
- Las fronteras de búsqueda se encuentran en la lista de nodos Open

Breadth - First

open = [H,I,J,K,L,M,N]

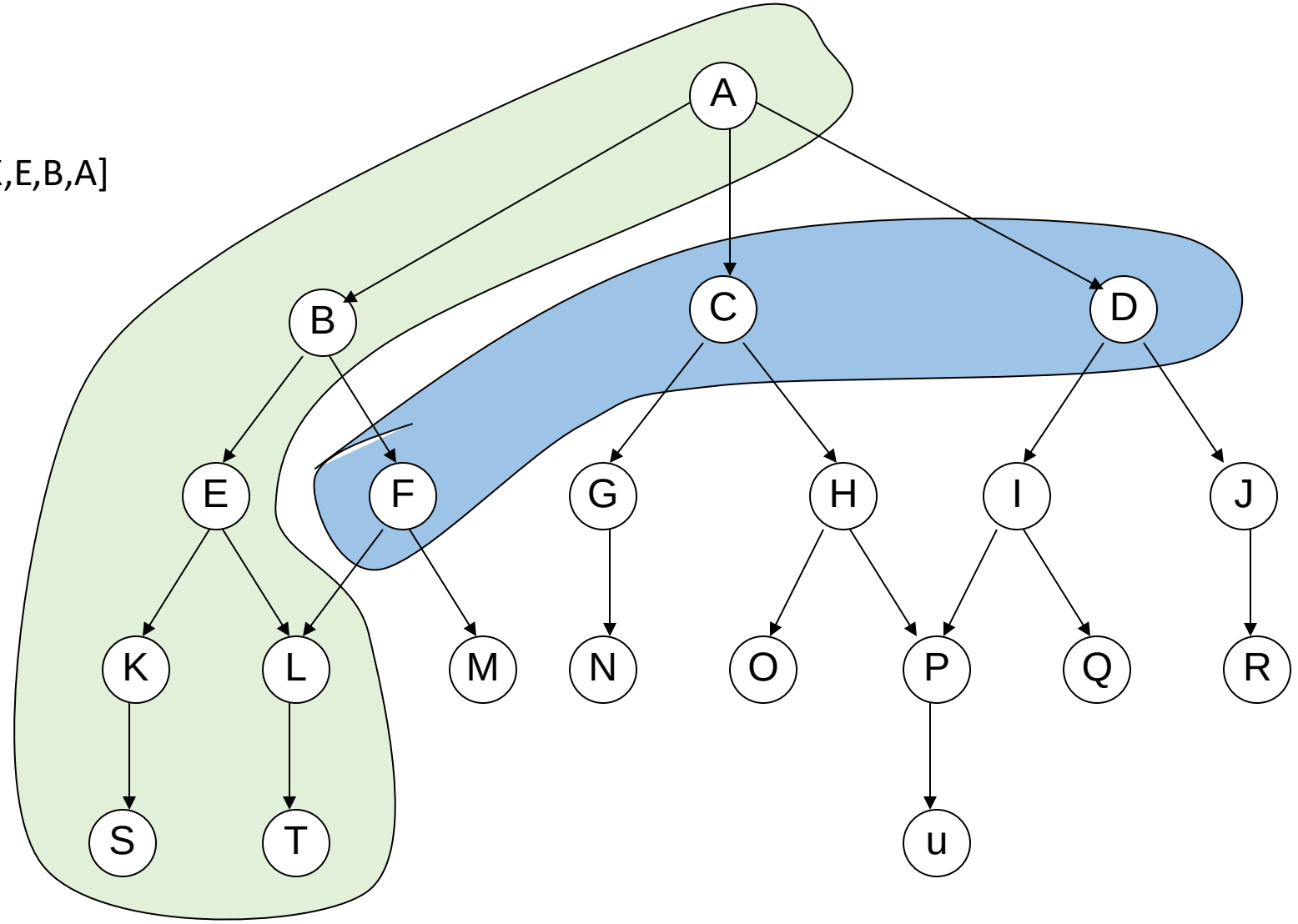
closed = [G,F,E,D,C,B,A]



Depth - First

open = [F,C,D]

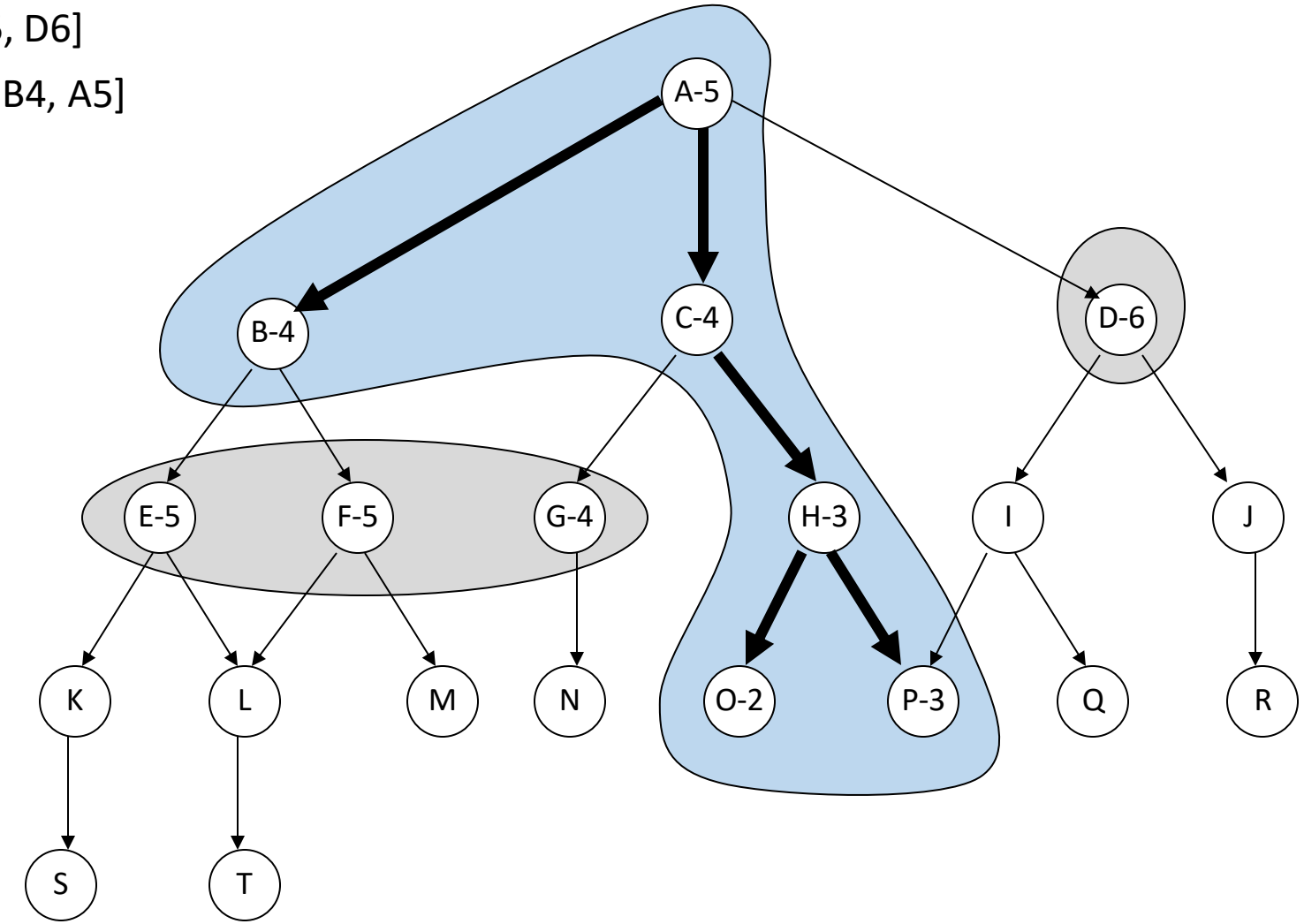
closed = [T,L,S,K,E,B,A]



Best - First

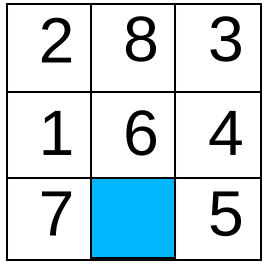
open = [G4, E5, F5, D6]

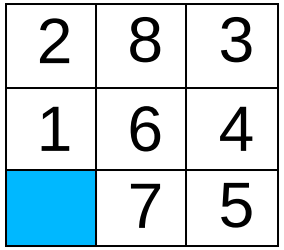
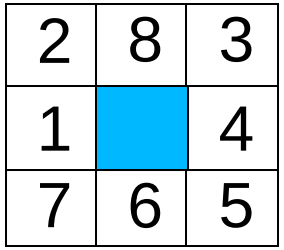
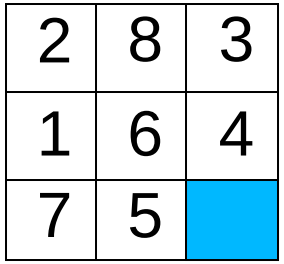
close [O2, H3, C4, B4, A5]



Búsqueda heurística: Ejemplo

Start



	5	6	0
	3	4	0
	5	6	0
h(n)	Tiles out of place	Sum of distances of the tiles out of place	2 x # of reversed tiles

Goal

